

SysMD @OMG

... Christoph Grimm, ...

TU Kaiserslautern
Chair of Cyber-Physical Systems; Chair of ...



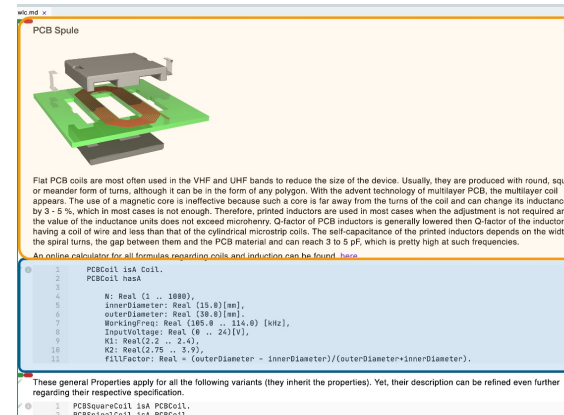
What is SysMD?

SysMD is a textual modeling language inspired by

- SysML v2: we use SysMLv2 metamodel (subset) & API for interoperability
- Markdown (MD): for document formatting of comments, description
- OWL & Turtle: (*Terse RDF Triple Language*) for semantic triple-like syntax

We distinguish:

SysMD language
(modeling &
documentation language)



SysMD notebook
(UI + compiler + analysis
framework)

Files hasA isA SysMD-Kickstart.md

SysMD-Kickstart.md

- > AnnElem: ScalarValues.md
- > Pkg: ScalarValues
- > AnnElem: Gbo.md
- > Pkg: GBO
- > AnnElem: Math.md
- > Pkg: Math
- > Pkg: Physics
- > AnnElem: SysMD-...
- > Def: Example isA ...
- Def: Vehicle isA Any
- Def: Car isA Vehicle
- Def: Bicycle isA Vehicle
- Def: VW isA Car
- Def: BMW isA Car
- ▼ Pkg: Vehicles
 - > Def: Vehicle isA Any
 - > Def: Car isA Vehicle
 - > Def: Bicycle isA Vehicle
 - > Def: VW isA Car
 - > Def: BMW isA Car
- ▼ Pkg: CarParts +
 - > Def: Body isA ...
 - > Def: Engine isA ...
 - > Def: Wheel isA ...
- > Pkg: SIUnits
- > Pkg: DateTime



SysMD Kickstart

For installation and use of the application have a look at the PDF file. It gives a detailed description of installation and the application. Notebook-elements are - like this one - natural language descriptions.

Documentation vs. Code-Elementes.

Markdown is *easy* to learn, efficient, and effective.

Heading

This is *emphasized*

Below an example for SysMD code.

Language: SysMD Scope: Global

```
1 Example isA Component.  
2 Example hasA
```

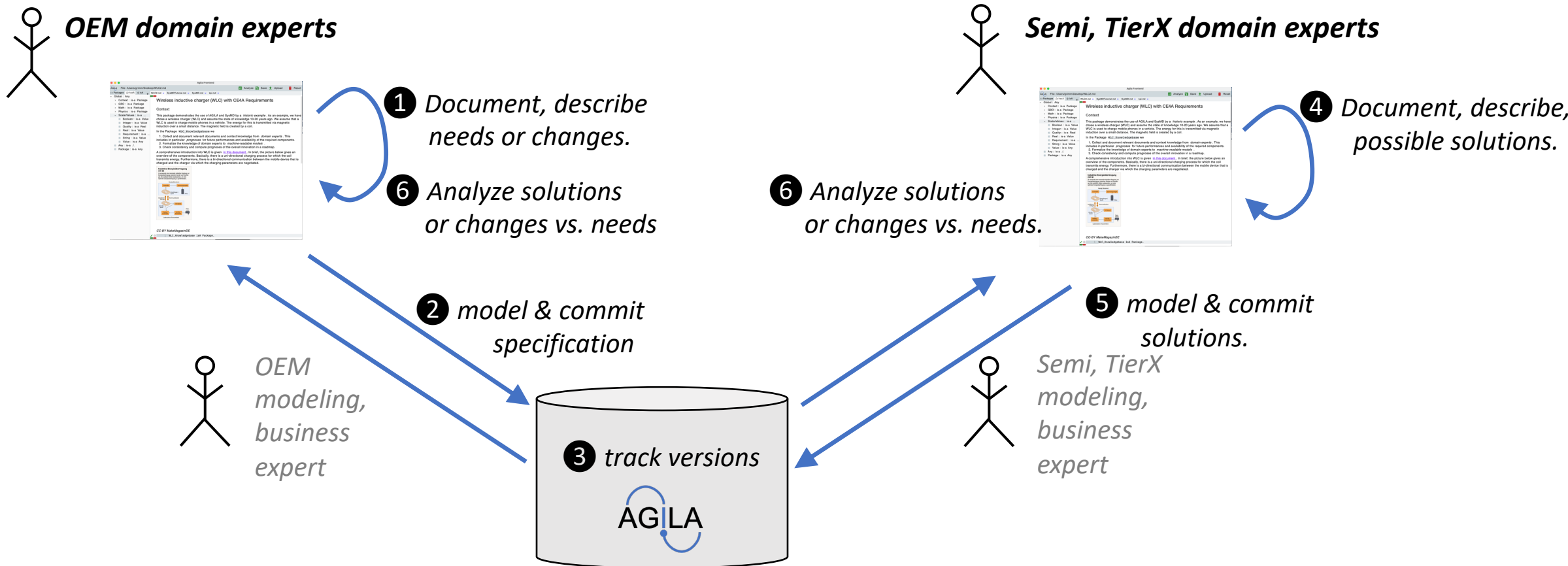
- Internal model implements subset of KerML Metamodel
- Frontend (UI) coupled to Backend (Graph-Database in Cloud) via REST
- Share & discuss experiences on
 - Ability to work interactively – impacts on SysML v2?
 - Experiences with use of metamodel; choice/definition of subset/profile?
 - Experiences with implementation of consistency checking / constraint satisfaction
 - Semantics of ranges, propagation of ranges, modeling of it in SysML metamodel
 - Implementation of REST API
 - ISSUE: identification in entities by ID? Or better Name?
 - BUG reports ;-) (ID still in keywords)

Contents

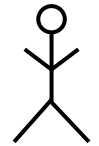


1. Introduction
2. SysMD Notebook
3. SysMD Language
4. Lessons learned for SysML v2
5. Summary

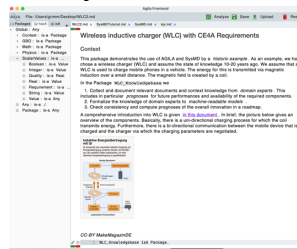
Use Case 1 (Agile Development; Technology Roadmapping)



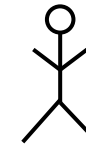
Use case 2 (Industrial DevOps with Runtime-Verification)



Developer@Development:
SysMD Notebook



① *Based on model & executable models: Derive constraints on signals.*

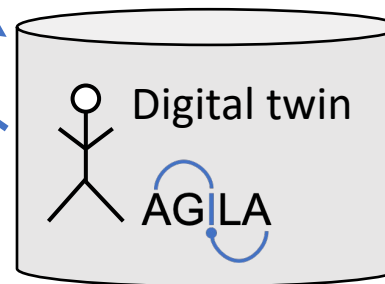


Deployed device@Runtime

- ③ *Observe signals and check: observed signal reachable by model?*
- ④ *Report event to digital twin.*

⑥ *update & analyze model; GOTO USE CASE 1.*

② *Deploy model & constraints in digital twin.*



⑤ *Label model with aggregated events.*

General context, assumptions, design decisions

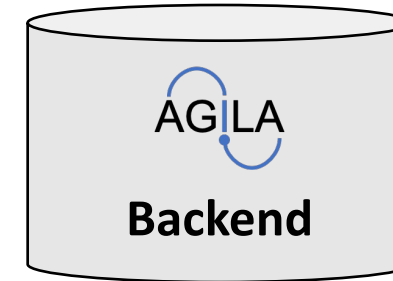
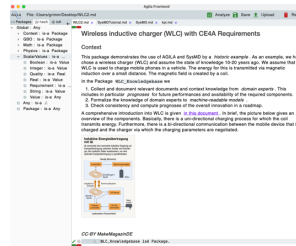
- 1) Inclusion of all stakeholders and communication of requirements/specs is needed
 - Stakeholders are often no (systems) modeling-experts
 - *Design decision*: Document-based view + model as annotation (not model + comments)
- 2) Communication across different domains requires explanations, disambiguation
 - *Design decision*: Integration of pictures, links, videos, etc. to explain
 - *Design decision*: Libraries or ontologies with standardized and broadly accepted vocabularies, i.e., ISO26262
- 3) Benefit beyond documentation
 - *Design decision*: Constraint-propagation checks consistency, helps in configuration values, design
 - *Perspective*: maintainable “expert system” with (hybrid) AI assists in specification and development
- 4) Version management in single place, e. g. cloud needed

Contents



1. Introduction: Contributions and Context
- 2. SysMD Notebook**
3. SysMD Language
4. Lessons learned
5. Summary

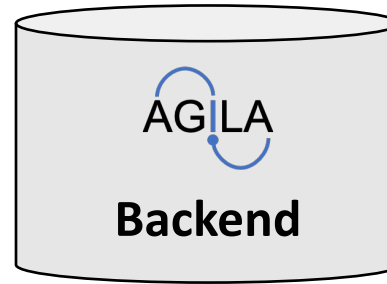
Overall System; Logical Overview



- **Documents** in Markdown, Videos, Pictures; hierarchically structured & linked
 - Annotated with related **models** in
 - SysMD (tightly linked with MD)
 - SysMLv2 Textual
- **Data** from user (spec, prognoses, ...)
- **Data** from verification/validation
- **Data** from operation

- **Instances from KerML metamodel classes**
 - Documentation (in SysMD)
 - TextualRepresentation (in language)
 - After compilation:
 - Elements, Relationships, ...
- Files (Video, Picture, ...)
- Versions

Overall System; Technical Overview



SysMD Notebook (web)

- Multiple users
- React JS; WiP

AGILA Backend

- Spring Boot application
- REST Controllers
- Version management
- **Constraint solver, reasoner**
- Repository: ArangoDB (Graph database)

Demo & presentation

SysMD Notebook (app)

- Jetpack Compose app

AGILA iDevOps

- Plugins for simulators
- Interfaces to sensors



- Digital twin data
- Reachability model



Deployed units

- Runtime-Verification
- Event detection

SysMD Notebook Overview

– Navigation in models

- › Projects, branches, commits
- › Decomposition: Project-Packages-Parts, Values, ...
- › Taxonomy: Any-Subclasses-Subsubclasses, ...

– Documentation of elements

- › Markdown-format
- › Tables, figures, links, ...

– Models in SysMD language

- › Definition of decomposition, taxonomy, instances,
- › Values, constraints, relationships

The screenshot displays the SysMD notebook interface. On the left, a package browser shows a hierarchy: Global: Any, Context: is-a Package, GBO: is-a Package, Math: is-a Package, Physics: is-a Package, ScalarValues: is-a ..., Boolean: is-a Value, Integer: is-a Value, Quality: is-a Real, Real: is-a Value, Requirement: is-a ..., String: is-a Value, Value: is-a Any, Any: is-a ./, and Package: is-a Any. The main editor shows a document titled 'Wireless inductive charger (WLC) with CE4A Requirements'. The document content includes a 'Context' section, a paragraph about the package's purpose, a list of three steps for knowledge collection and formalization, and a paragraph about the introduction to WLC. Below the text is a diagram titled 'Induktive Energieübertragung mit Qi' showing the components of a wireless charger: Handy (Receiver), Spannungsgemäßer, Akku, Induktion/Energie, Kommunikation, V77-Sens, Treiber/Controller, AC/DC-Konverter, and Netz-kabel. The bottom status bar shows 'CC-BY MakeMagazinDE' and '1 WLC_Knowledgebase isA Package.'

Agila Frontend

File: /Users/grimm/Desktop/WLC2.md

Analyze Save Upload Reset

Packages hasA isA WLC2.md SysMDTutorial.md SysMD.md kpi.md

Wireless inductive charger (WLC) with CE4A Requirements

Context

This package demonstrates the use of AGILA and SysMD by a *historic example*. As an example, we have chose a wireless charger (WLC) and assume the state of knowledge 10-20 years ago. We assume that a WLC is used to charge mobile phones in a vehicle. The energy for this is transmitted via magnetic induction over a small distance. The magnetic field is created by a coil.

In the Package `WLC_Knowledgebase` we

1. Collect and document relevant documents and context knowledge from *domain experts*. This includes in particular *prognoses* for future performances and availability of the required components.
2. Formalize the knowledge of domain experts to *machine-readable models*.
3. Check consistency and compute prognoses of the overall innovation in a roadmap.

A comprehensive introduction into WLC is given [in this document](#). In brief, the picture below gives an overview of the components. Basically, there is a uni-directional charging process for which the coil transmits energy. Furthermore, there is a bi-directional communication between the mobile device that is charged and the charger via which the charging parameters are negotiated.

Induktive Energieübertragung mit Qi

Qi verwendet eine resonante induktive Kopplung zur Energieübertragung zwischen Sender und Empfänger, die zusätzlich Daten austauschen, um eine optimale Energieübertragung zu gewährleisten.

Handy (Receiver)

Controller Spannungsgemäßer Akku

(Empfänger-) Spule

Induktion/ Energie Kommunikation

(Sender-) Spule V77-Sens

Treiber/ Controller AC/DC-Konverter Netz-kabel

Ladestation (Transmitter)

CC-BY MakeMagazinDE

1 WLC_Knowledgebase isA Package.

(further windows for results of analysis)

SysMD language vs. SysML v2 textual

SysML v2 (textual)

- Based on KerML, SysML API
- Syntax close to programming languages.
- Target: modeling and SE experts.
- Documentation added to model.
- Expressions allow modeling of constraints, spec. 

```
Wheel {  
  value mass: Real = 70 [kg];  
  // model mass with 50 to 100 kg  
}  
  
Car :> Vehicle {  
  part Wheel [4 .. 8];  
  in value mass = ... // model constraint, unit, ...  
}
```

SysMD

-  • Based on KerML, SysML API (BDD, IBD; deviations are “Bug”).
-  • Closer to **natural language**.
-  • Target: users are **domain experts**.
-  • Model added to **documentation** (text, videos, ...).
-  • Syntax separates **specification** and **modeling**.

```
Car isA Vehicle.  
  
Car hasA  
  wheel: [4 .. 8] Wheel,  
  mass: Mass(100..1000) kg = sumHasA(mass).  
  
Wheel hasA  
  mass: all Mass(50 .. 100) kg = ....
```

Bi-Directional Constraint Propagation in SysMD

- Direct dependencies given by expressions
 - Constraint propagation on Reals, Integers;
 - Checks and converts Units (incl. nat. units, dB, Date/Time) and Domains
 - Satisfiability problem for Booleans
- Inheritance
 - Models variants or potential solutions of similar things
 - Consistency check; choice of matching subcl.
- Decomposition
 - Models parts of a thing
 - Constraint propagation constrains cardinality $\leftrightarrow$ values

```
1 Example isA Component.  
2 Example hasA  
3   height: Real(10 .. 100)[cm],  
4   width:  Real(1 .. 1.1) [m],  
5   length: Real(1 .. 1.1) [m],  
6   volume: Real(1 .. 2) [m^3] = height * width * length.
```

```
1 Vehicles::Car hasA power: Real(10 .. 1000) [kW].  
2 Vehicles::VW  hasA power: Real(20 .. 1010) [kW].  
3 Vehicles::BMW hasA power: Real(150 .. 400) [kW].
```

Vehicles::Car::power = 10..1000 kW

Vehicles::VW::power = 20..1010 kW

Vehicles::BMW::power = 150..400 kW

INFO in Vehicles::Vehicle: different units in different subclasses

ERROR in Vehicles::VW::power: INCONSISTENCY: subclass value 20..1010 of power must be refinement of superclass value 10..1000

```
1 Vehicles::Car hasA  
2   body: CarParts::Body,  
3   wheels: [4 .. 4] CarParts::Wheel,  
4   engine: [1 .. 2] CarParts::Engine,  
5   mass: Real [kg] = SUM(mass).
```

Vehicles::Car::mass = 500..700 kg

Contents



1. Introduction: Contributions and Context
2. SysMD Notebook
- 3. SysMD Language**
4. Lessons learned
5. Summary

Language Design Objectives

Workflow in “Notebook”: enriching documentation with models (not other way around)

- Small pieces of code possible that can be compiled separately; incremental compiling
- Possibility of execution/evaluation of incomplete models

Readability

- Arbitrary ordering of definitions, statements, ... -- in particular “top-down”
- Distinguish specification from computation artefacts in models
- Syntax that re-uses small number of patterns, close to natural language

Interoperability with SysML v2, OWL

- Translation to SysMLv2 metamodel (maybe roundtrip with SysMLv2 Textual?)

Language Basics - Lexical elements

- **Keywords**
 - isA, hasA, defines, imports,
 - one, all, and, or, Project
- **Literals** (like SysML v2 textual; deviation is error if not mentioned explicitly)
 - *String, Real, Integer, Boolean* (“true”, “false”),
 - *Name*
- **Operators**
 - +, -, *, /, ^, <, <=, ==, >, >=, .. in expressions
 - ::, . for qualified names, feature chains
- **Comments**
 - Lexical comments start with “//” and end with the line.
 - “Comments++” are written & rendered by Markdown (MD); “Code” handled by SysMD

Examples:

Valid String literal

- “this is a String”

Valid Real literal

- 24.5E-5

Valid Name literals

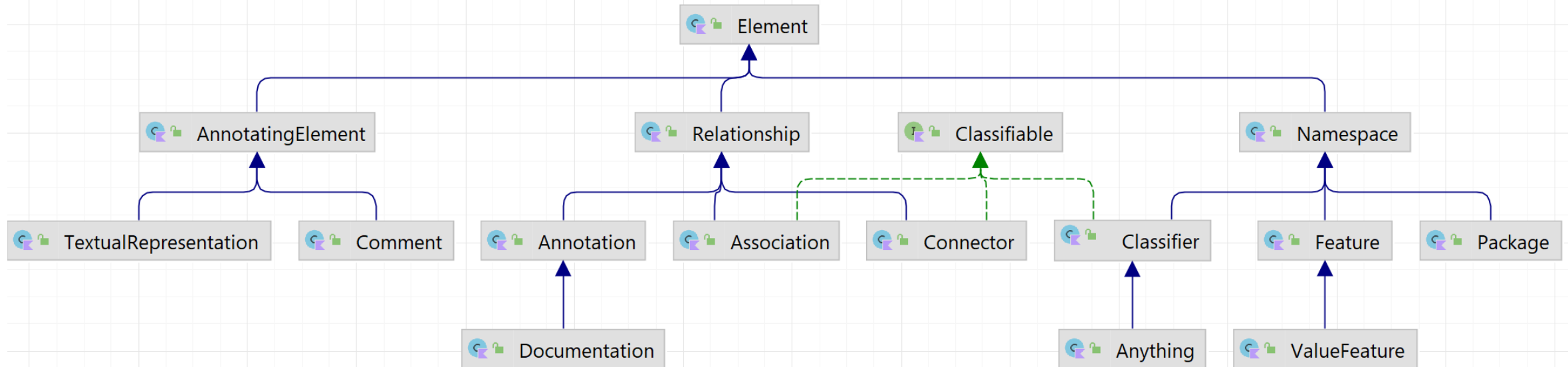
- a_b_c123
- ‘a b c’
- < ‘1’ >

Subject	Predicate	Object	More objects	Ends with
<i>Name</i> <Element>	defines	<i>Name isA Name</i> <Classifiable or Instantiable>	(; Definition)*	.
	hasA	<i>Name</i> : [all one] [Multiplicity] <i>Name</i> <Type> [Constraints] [= Expression]	(, Name)*	
	imports	<i>Name</i> <Project, Namespace>	(, Name)*	
	<i>Name</i> <Association>	<i>Name</i> <Element>	(, Name)*	

Pre-defined classes and projects

- ScalarValues (Classifies Real, Boolean, Integer, ... as in SysMLv2)
- ISO26262 Ontology: Element, Function, Component, Part, SoftwareUnit, (...), also relationships:
Component *implements* Function, Component *satisfies* Requirement, Processor *executes* Software
- GBO, MissionProfiles, Math, Physics.

SysMD Semantics - Mapping to SysMLv2 Metamodel & Constraint Satisfaction Problem



Note:

- 1) We are not yet fully compatible ... working on it.
- 2) We strive to consolidate number of classes a bit.
(e.g., ValueFeature includes Expression, Multiplicity, FeatureValue, ...)
- 3) We strive to increase performance, reduce complexity – not all relationships represented by instances of Relationship (e.g. ownership, inheritance)

“defines” introduces Classes, Packages

Subject	Predicate	Object	More objects	Ends with
<i>Name</i> <Namespace>	defines	<i>Name isA Name</i> <Classifiable, Package>	(; Definition)*	.

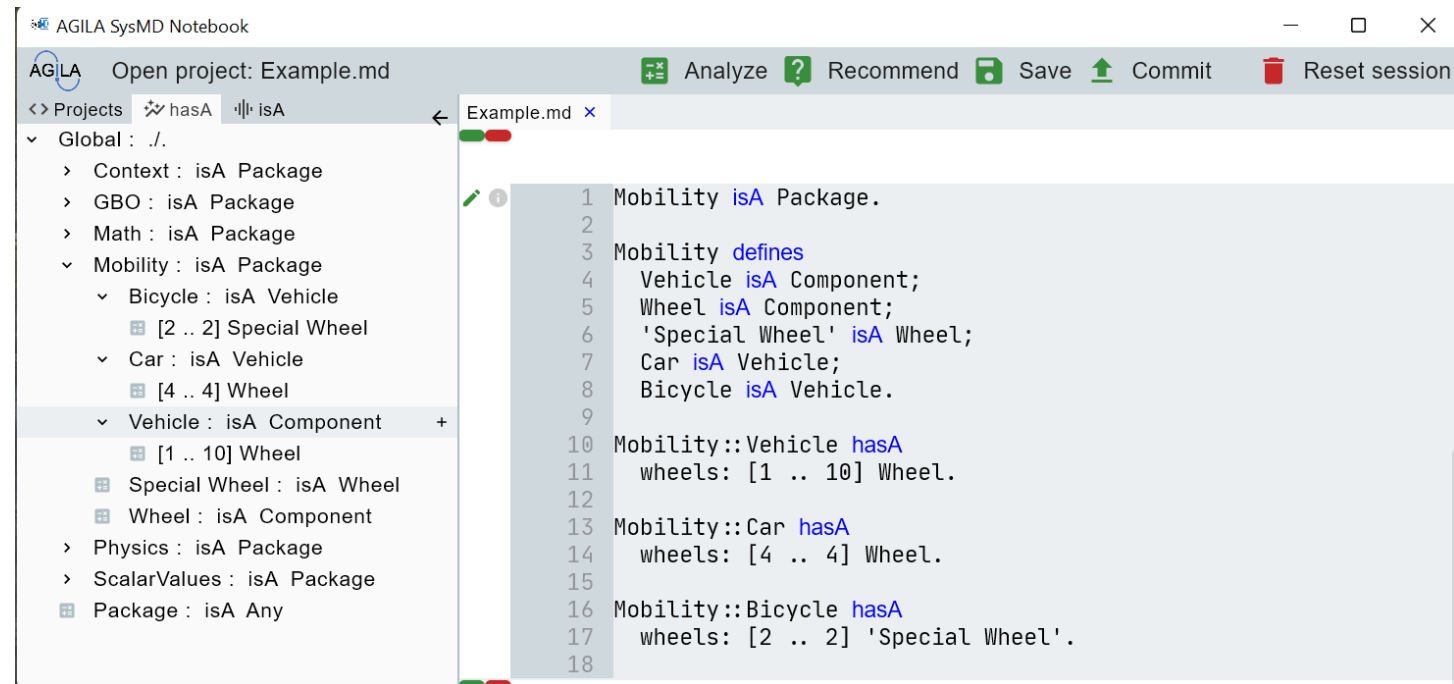
Compiler adds

- *Classifiable* class,
- *Package*

to Namespace of Subject

Consistency checks

- Does Liskov principle hold?



The screenshot shows the AGILA SysMD Notebook interface. On the left, a project tree for 'Example.md' is visible, showing a hierarchy of packages and components. The 'Global' package contains 'Context', 'GBO', and 'Math'. The 'Mobility' package contains 'Bicycle', 'Car', and 'Vehicle'. The 'Vehicle' package contains 'Wheel' and 'Special Wheel'. The 'Physics' package contains 'ScalarValues'. The 'Package' package contains 'Any'. On the right, a code editor shows the following code:

```
1 Mobility isA Package.
2
3 Mobility defines
4   Vehicle isA Component;
5   Wheel isA Component;
6   'Special Wheel' isA Wheel;
7   Car isA Vehicle;
8   Bicycle isA Vehicle.
9
10 Mobility::Vehicle hasA
11   wheels: [1 .. 10] Wheel.
12
13 Mobility::Car hasA
14   wheels: [4 .. 4] Wheel.
15
16 Mobility::Bicycle hasA
17   wheels: [2 .. 2] 'Special Wheel'.
18
```

Classification and inheritance

SysMD uses inheritance to model alternatives or variants

- E.g., two variants of one thing.

SysMD inheritance

- Currently restricted to **single inheritance**;
multiple inheritance when we are sure about implications.
- For semantics and constraints, the **Liskov principle** seems natural requirement:
A specialization shall be able to replace a general class.

Hence

- Subclasses may only **extend** functionality (e.g., add functions, parts, values)
- Subclasses may only **constrain** ranges of values with **one-range** (e.g., represent specific solution)
- Subclasses may only **extend** ranges of values with **all-range** (e.g., permit wider temperature range)

→ SysMDTutorial.md ✕

A subclass inherits the values and relationships of its superclass. A subclass can

- reduce the range of values,
- add properties, but not have less properties.
- add relationships, but not have less relationships.

```
1 Inheritance isA Package.  
2 Inheritance defines  
3   General isA Component;  
4   Subclass isA General.  
5  
6 Inheritance::General hasA  
7   p: Real(1 .. 10).  
8  
9 Inheritance::Subclass hasA  
10  p: Real(2 .. 5).  
11  
12 Display shows Inheritance::General, Inheritance::Subclass.
```

In General:

p = 1..10

In Subclass:

p = 2..5

→ SysMDTutorial.md

A subclasse inherits the values and relationships of its superclass. A subclass can

- reduce the range of values,
- add properties, but not have less properties.
- add relationships, but not have less relationships.

```
1 Inheritance isA Package.  
2 Inheritance defines  
3   General isA Component;  
4   Subclass isA General.  
5  
6 Inheritance::General hasA  
7   p: Real(1 .. 10).  
8  
9 Inheritance::Subclass hasA  
10  p: Real(2 .. 20).
```

```
12 Display shows Inheritance::General, Inheritance::Subclass.
```

ERROR: In property p: column 1: INCONSISTENCY: subclass value 2..20 of p must be refinement of superclass value 1..10

In General:
p = 1..10

“hasA” adds Features, Expressions

Subject	Predicate	Object	More objects	Ends with
<i>Name</i> <Namespace>	hasA	<i>Name</i> : [all one] [Multiplicity] <i>Name</i> <Type> [Constraints] [“=” Expression]	(, Name)*	.

Compiler adds to Namespace (Subject)

- For each object, a *Feature* with Direction: *out* (all), in (one)
Expression: (by AST)
Constraints: (by Constant AST)

Consistency checks:

- Are constraints satisfiable?
 - For (at least one | all) valuations?

Specific support for “Constraint Propagation”

Name “:” [all|one] *TypeIdent* [Restriction] [“=” Expression]

a: Real(1..2) = b+c;

a: Integer(1..2) = b+c;

a: Boolean(false) = b or c;

SysMD assumes all Real/Integer are restricted to and represented by Ranges (intervals)

- Constraint propagation likes small intervals
- Technically, we anyhow need to deal with intervals – floating point is not Real, no pi, ... (!)

In SysMD we suggest users to

- SPECIFY upper and lower bounds
 - is also good engineering practice: concise, brief, easy to find out meaning compared with coding constraints into expressions.
 - Significantly helps CSP solver.

... ONE more thing ;-)

Name ":" [**all** | **one**] *TypeIdent* [Restriction] ["=" Expression]

a: **all** Real(1..2) = b+c;

Following expression must be satisfied for **all values** of the specified range.
i.e., a random value from the environment for which (all!) some implementation must fit.

a: **one** Real(1..2) = b+c;

Following expression must be satisfied for **at least one value** of the specified range.
i.e., a value of the design that we want to pick from a given range.

Values are Quantities with Unit and Dimension!

Name ":" [all|one] *TypeIdent* [Restriction] ["=" Expression]

a: Width(1..2) [m]= 10 m + 5 cm + 10 [km^2/m]

b: Length, c: Height

- Specified unit refers to the specified range. Required iff range is given.

a: Real(1..2) [m]= 10 m + 5 cm + 10 [km^2/m]

- Simple units can directly follow a number literal; expressions of units must be enclosed in[]
- SysMD tracks, converts units (SI, national units, %, ...), reports inconsistencies.

→ SysMDTutorial.md x

Dependencies with Units

Simple dependencies

Dependencies between properties can be modeled by using boolean or arithmetic expressions.

```

1 Dependencies isA Package.
2 Dependencies defines component isA Component.
3 Dependencies::component hasA
4   d: Real(1.0 .. 20.0)[mm],
5   c: Real(2.0 .. 3.0)[m],
6   b: Real(1.0 .. 2.0)[km],
7   a: Real = b*c*d.
8 Display shows Dependencies::component.

```

— Play with the values and the formula. Also try to restrict a by giving it concrete values or ranges, for example `a: Real(120 m^3)`.

As can be seen above, Appel supports units. The units can be specified after each number literal and with each type declaration. Consistency is checked, and units are converted for internal solving into SI units.

```

1 Units isA Package.
2
3 Units hasA
4   t: Real [s] = 1.0 [s],
5   v: Real [m/s] = 3.0 [m/s],
6   g: Real [m/s^2] = 4.0 [m/s^2],
7   s: Real [m/s] = sqrt(sqr(v)+sqr(g)*sqr(t)).
8
9 Display shows Units

```

Contents



1. Introduction
2. SysMD Notebook
3. SysMD Language
- 4. Lessons learned for SysML v2**
5. Summary

Lessons learned regarding SysMLv2 Metamodel



- We could map SysMD models to metamodel (and, to be shown, from there to SysMLv2)
 - The SysMLv2 metamodel looks complete
 - *How do we differentiate clearly between instances and classes?*
- The LRM gives a mostly clear specification of the metamodel
 - However, it is often hard to read; would prefer mapping syntax to changes in Metamodel
 - Partially incomplete
 - Marking missing pieces as *t.b.d.* would make clearer “really” missing
 - Smaller inconsistencies (e. g. “id” is no longer needed, but still listed keyword)
- Efficient implementation
 - Metamodel is good for data base queries, exchange of models
 - Important relations like ownership, superclass profit in implementation from less modeling.
 - For ownership, superclass, expressions we use references (pointers); no relationships

- The Rest API
 - Use case of **exporting** models via REST and version management is covered
 - How is identification when ensured? Via UUID? Or qualified name?
- BUT: Import? Tool coupling?
 - SysMD has Web-Frontend with Session-Service, document management, constraint propagation
 - Import or Elements of Metamodel (easy, basically post of elements)?
 - *Synchronization of basic status* of tools (without creating new commit)
 - Save/post source code
 - Synchronize changes of Values (Results) in both directions
 - Report errors

GENIAL!/SysMD Profil: *Explicit Specification*

We make specified values explicit and distinguish them from modeling artefacts.

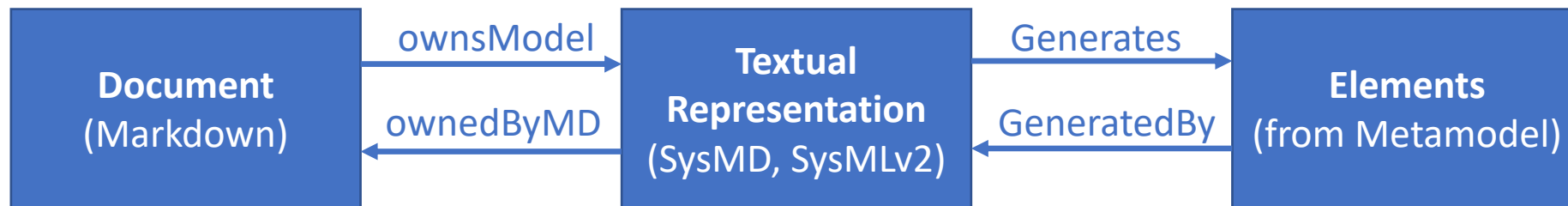
```
volume: Volume(1.0 .. 2.5) [cm^3] = width * length * height.
```

- Class ValueFeature combines metamodel elements
 - Name (volume)
 - Classifier (Volume); the domain
 - Value (1.0 .. 2.5); the specified value
 - String/SIUnit (cm^3); the unit used in user view
 - Expression width * length * height; dependency that must hold for
 - Direction of Feature all or at least one of the specified values
- Not yet represented in metamodel, but should be possible
- Maybe loss of information when roundtrip with SysMLv2 textual

GENIAL! Profil: Linking Document, Model, Element

For *tracability from document to model to element*, we

- Start with Documents (in Markdown)
 - Also, hierarchical documents with links, references across them.
- Add Textual Representations of Models as owned elements to the Documents
- Compiler adds Metamodel elements to root
 - Association “GeneratedBy” / “Generates” links Textual Model with metamodel elements



Contents



1. Introduction
2. SysMD Notebook
3. SysMD Language
4. Lessons learned for SysMLv2
5. **Summary**

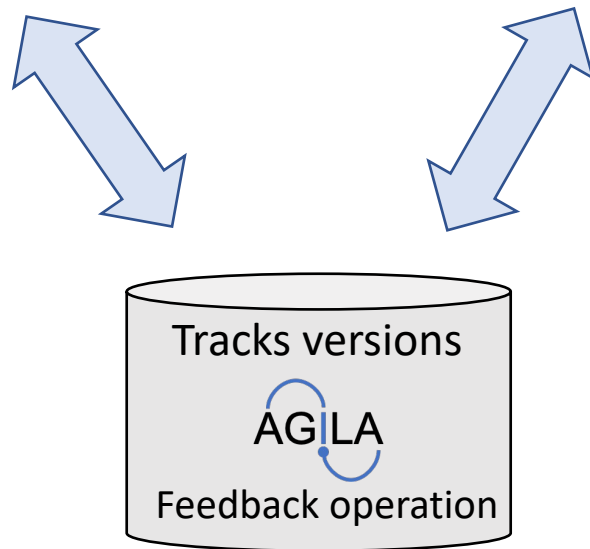
Implementation status



Still many issues and bugs 😞
UI just ok ... but must improve.

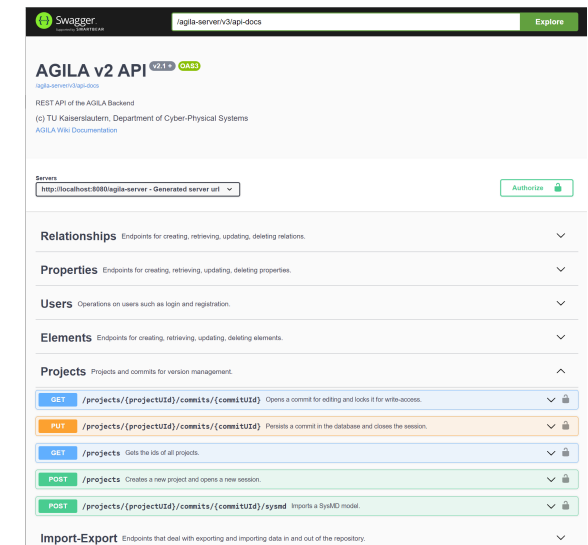
Feature in progress:

- Probabilistic properties.
- Runtime-Verification running, needs integration



Implementation (Agila & SysMD)

- **Rest API** (we strive for compatibility with SysML for subset, not 100%)
- SpringBoot, ArangoDB
- Kotlin implementation
- Jetpack Compose for UI
- Junit Jupiter tests (more than 1000 tests)



Outlook

- 1st Release to public (open source) mid or end 2022
 - Basically, as shown today, but with less bugs & some libraries
 - Open source for most parts
 - Small parts or patent pending, but not necessarily needed
- 2nd Release end 2023/2024
 - Integrated DevOps interface
 - Automated generation of interfaces to virtual prototypes
- Search for collaborations with industry; new projects